Unit Testing Intermediate

Zephyr RTOS Intermediate Course



Quick Recap — Lecture 7

- EU regulations: device security is now mandatory
- Security in Zephyr: isolation, crypto and keys, vulnerability handling, `hardenconfig`
- CRC vs hash vs signature vs encryption
- Secure boot: a chain of trust from a hardware anchor
- Secure boot in Zephyr: MCUboot via sysbuild, or BL2 with TF-M
- Live QEMU demo: the valid image boots, the tampered and wrong-key images are rejected

Recap of Basic Course: Unit Testing

- **Ztest** — `ZTEST_SUITE` with its six arguments, `ZTEST`, the `before()` hook
- **Assertion families** — `zassert_*` stops, `zexpect_*` records, `zassume_*` skips
- **Twister** — discovers `testcase.yaml`, builds per platform, parses the output
- `twister-out/` — logs, reports, and a `zephyr.exe` you can run under gdb
- **Coverage** — `--coverage --coverage-tool gcovr --gcov-tool gcov --coverage-basedir <module>`



Today's Agenda



- The dependency problem — why plain Ztest stops working
- Which calls can be faked — and which cannot
- **FFF** — `return_val`, `call_count`, `custom_fake`, `arg<N>_val`, sequences
- Reset discipline — fakes are global state
- What mocking unlocks: **error-path coverage**
- When FFF cannot reach: faking through a function pointer
- Homework



The Dependency Problem



The Module Under Test: `calculator_nvs`



A calculator module with one new job: **persist every result**.

```
/* app/modules/calculator_nvs/include/calculator_nvs.h */
int  calculator_nvs_init(struct nvs_fs *fs);
void calculator_nvs_deinit(void);
int  calculator_nvs_add(int a, int b, int *result);
int  calculator_nvs_sub(int a, int b, int *result);
int  calculator_nvs_get_last_result(int *result);
```

Internally it calls exactly three functions: `nvs_mount()`, `nvs_write()`, `nvs_read()`.

Why the Real Dependency Is the Enemy

What testing against real NVS costs	What you cannot test at all
Needs a flash device and a partition	`nvs_write()` returning `-EIO`
Needs devicetree wiring per board	`nvs_write()` accepting only 1 of 4 bytes
State survives between tests	`nvs_mount()` failing at startup
Slow, and shared with the rest of the system	Storage full, mid-operation



FFF Fundamentals



What is FFF?



FFF = Fake Function Framework — a small set of C macros for writing fakes.

- Integrated into Zephyr: `#include <zephyr/fff.h>`
- One macro creates the fake function and a struct to control it
- You set what the fake returns, and it records how it was called
- Used in Zephyr's own unit tests and in the `zephyr,fake-*` drivers



Which Calls Can Be Faked?



A fake works by **being the only definition of the function in the build**.

Call shape	Can FFF replace it?
Ordinary extern function — `nvs_write()`	**Yes** — you provide the symbol
`static` function inside the module	No — there is no symbol to replace
`static inline` in a header	No — inlined into the caller
`__syscall` wrapper (most driver APIs)	No — expands to a `static inline`
Macro — `LOG_INF()`	No — gone before the compiler sees a call
Anything the compiler inlined via LTO	No — there is no call left



One Macro, One Fake




```
#include <zephyr/fff.h>

FAKE_VALUE_FUNC(int, nvs_mount, struct nvs_fs *);
```

Creates the function `nvs_mount()` and the struct `nvs_mount_fake`:

Field	Meaning
`.return_val`	What the fake returns
`.call_count`	How many times it was called
`.arg<N>_val`	Nth argument of the last call (0-indexed)
`.custom_fake`	Function to run instead



Header and Implementation: `DECLARE` / `DEFINE`




```c
/* tests/calculator_nvs/include/nvs_fake.h */
DECLARE_FAKE_VALUE_FUNC(int, nvs_mount, struct nvs_fs *);
DECLARE_FAKE_VALUE_FUNC(ssize_t, nvs_read, struct nvs_fs *, uint16_t, void *, size_t);
DECLARE_FAKE_VALUE_FUNC(ssize_t, nvs_write, struct nvs_fs *, uint16_t, const void *, size_t);
```

```c
/* tests/calculator_nvs/src/nvs_fake.c */
DEFINE_FFF_GLOBALS;   /* exactly once per test binary */

DEFINE_FAKE_VALUE_FUNC(int, nvs_mount, struct nvs_fs *);
DEFINE_FAKE_VALUE_FUNC(ssize_t, nvs_read, struct nvs_fs *, uint16_t, void *, size_t);
DEFINE_FAKE_VALUE_FUNC(ssize_t, nvs_write, struct nvs_fs *, uint16_t, const void *, size_t);
```



Wiring the Build




```
target_sources(app PRIVATE
    src/nvs_fake.c
    src/test_basic.c
    src/test_error_paths.c
    ${CMAKE_CURRENT_SOURCE_DIR}/../../app/modules/calculator_nvs/src/calculator_nvs.c
)
```

```
CONFIG_ZTEST=y
CONFIG_ZTEST_SHUFFLE=y
CONFIG_LOG=y
# CONFIG_NVS is NOT enabled on purpose
```



`return_val` — What the Fake Returns




```
ZTEST(nvs_init, test_mount_failure)
{
    /* return_val: every call returns this value */
    nvs_mount_fake.return_val = -EIO;

    int ret = calculator_nvs_init(&fake_fs);

    zassert_equal(ret, -EIO, "mount error must be returned unchanged, got %d", ret);
}
```

- Set once, every subsequent call returns it
- Back to `0` after a reset



`call_count` — Did the Call Happen?




```
ZTEST(nvs_init, test_null_fs_pointer)
{
    int ret = calculator_nvs_init(NULL);

    zassert_equal(ret, -EINVAL, "NULL pointer must return -EINVAL, got %d", ret);

    /* The guard must return before the driver is called */
    zassert_equal(nvs_mount_fake.call_count, 0,
                  "nvs_mount must not be called when fs is NULL, got %u calls",
                  nvs_mount_fake.call_count);
}
```

- `call_count` records how many times the fake ran



DEMO 1: Your First Fake



`custom_fake` — When a Return Value Is Not Enough




```c
/* tests/calculator_nvs/src/nvs_fake.c */
static ssize_t custom_nvs_write(struct nvs_fs *fs, uint16_t id, const void *data, size_t len)
{
    /* ... ARG_UNUSED, length check ... */
    simulated_nvs_storage = *(const int *)data;   /* capture while it is live */
    has_data = true;
    return (ssize_t)len;
}
```



`custom_fake` — When a Return Value Is Not Enough




```c
/* tests/calculator_nvs/src/nvs_fake.c */
void nvs_fake_init(void)
{
    nvs_read_fake.custom_fake  = custom_nvs_read;
    nvs_write_fake.custom_fake = custom_nvs_write;
}
```



`arg<N>_val` — and Pointer Arguments




```
/* nvs_write(fs = arg0, id = arg1, data = arg2, len = arg3) */
zassert_equal(nvs_write_fake.arg1_val, CALCULATOR_NVS_ID,
              "the module must write to entry id %u, got %u",
              CALCULATOR_NVS_ID, nvs_write_fake.arg1_val);
```

- Zero-indexed — `arg0_val` is the first argument
- Holds the most recent call only
- Not for pointer arguments — capture the value inside `custom_fake` instead



`fff.call_history` — the Order of Calls




```
zassert_equal(fff.call_history[0], (fff_function_t)nvs_mount,
              "the first recorded call should be nvs_mount");
zassert_equal(fff.call_history[1], (fff_function_t)nvs_write,
              "add() must write before anything reads");
zassert_equal(fff.call_history[2], (fff_function_t)nvs_read,
              "get_last_result() must be the third recorded call");
```

- One array, shared by all fakes
- Proves one fake ran before another



`arg<N>_history` — Every Call's Arguments




```
zassume_ok(calculator_nvs_add(1, 1, &result), "precondition: add must succeed");
zassume_ok(calculator_nvs_sub(9, 4, &result), "precondition: sub must succeed");

zassert_equal(nvs_write_fake.arg1_history[0], CALCULATOR_NVS_ID,
              "the first write must use entry id %u", CALCULATOR_NVS_ID);
zassert_equal(nvs_write_fake.arg1_history[1], CALCULATOR_NVS_ID,
              "every write must use the same entry id");
```

- One array per argument, one entry per call
- Proves what earlier calls passed, not just the last



`SET_RETURN_SEQ` — a Different Value per Call




```
static ssize_t seq[] = {-EIO, -ENOSPC};

nvs_write_fake.custom_fake = NULL;      /* clear custom_fake, or the sequence is ignored */
SET_RETURN_SEQ(nvs_write, seq, ARRAY_SIZE(seq));

zassert_equal(calculator_nvs_add(1, 0, &result), -EIO, "call 1: seq[0] is -EIO");
zassert_equal(calculator_nvs_add(2, 0, &result), -ENOSPC, "call 2: seq[1] is -ENOSPC");
zassert_equal(calculator_nvs_add(3, 0, &result), -ENOSPC,
              "call 3: an exhausted sequence repeats its last element");
```



`SET_CUSTOM_FAKE_SEQ` — a Different Function per Call




```c
static ssize_t write_ok(struct nvs_fs *fs, uint16_t id, const void *data, size_t len)
{
    stored = *(const int *)data;       /* a side effect: SET_RETURN_SEQ cannot do this */
    return (ssize_t)len;
}

static ssize_t write_short(struct nvs_fs *fs, uint16_t id, const void *data, size_t len)
{
    return 1;                          /* flash accepted only 1 of 4 bytes */
}

static ssize_t (*write_seq[])(struct nvs_fs *, uint16_t, const void *, size_t) = {
    write_ok, write_short,
};
```



`SET_CUSTOM_FAKE_SEQ` — a Different Function per Call




```
SET_CUSTOM_FAKE_SEQ(nvs_write, write_seq, ARRAY_SIZE(write_seq));  /* outranks custom_fake */

zassert_ok(calculator_nvs_add(2, 3, &result), "call 1: write_ok stores 5");
zassert_equal(calculator_nvs_add(1, 1, &result), -EIO, "call 2: a short write is -EIO");
zassert_equal(calculator_nvs_add(4, 4, &result), -EIO, "call 3: repeats the last function");
zassert_equal(stored, 5, "call 1 must have written 5");
```



Precedence: Who Wins?



When a fake is called, FFF checks in this order:

1. `custom_fake_seq` — a sequence of functions, set by `SET_CUSTOM_FAKE_SEQ`
2. `custom_fake` — a single function
3. `return_val_seq` — a sequence of values, set by `SET_RETURN_SEQ`
4. `return_val` — a single value

First one that is set wins. The rest are silently ignored.



DEMO 2: Beyond `return_val`



Reset Discipline



Resetting a Fake



Macro	What it clears
`RESET_FAKE(name)`	One fake: `call_count`, `arg<N>_val`, return values, **and** `custom_fake`
`FFF_RESET_HISTORY()`	The call history shared by all fakes

- After a reset, install `custom_fake` again
- Your own simulated state is not covered — reset it yourself



Fakes Are Global Mutable State




```
void nvs_fake_reset(void)
{
    RESET_FAKE(nvs_mount);          /* per-fake: counts, args, return values */
    RESET_FAKE(nvs_read);
    RESET_FAKE(nvs_write);
    FFF_RESET_HISTORY();            /* global: the shared call_history array */

    simulated_nvs_storage = 0;      /* your own simulated state */
    has_data = false;
}
```



Fakes Are Global Mutable State




```
static void nvs_ready_before(void *fixture)
{
    nvs_fake_reset();               /* wipe everything */
    nvs_fake_init();                /* re-install the custom fakes */
    zassume_ok(calculator_nvs_init(&fake_fs), "precondition: init must succeed");
}
```



DEMO 3: When a Fake Outlives Its Test



What Mocking Unlocks



Where the Error Paths Hide

Zephyr®

```c
static int calculator_nvs_save(int result)
{
    if (ctx_fs == NULL) {
        return -EACCES;                    /* never: init always ran */
    }

    ssize_t rc = nvs_write(ctx_fs, CALCULATOR_NVS_ID, &result, sizeof(result));
    if (rc < 0) {
        return (int)rc;                    /* never: flash does not fail */
    }

    if (rc != (ssize_t)sizeof(result)) {
        return -EIO;                       /* never: no short writes */
    }
    return 0;
}
```



Error-Path Coverage: Real Numbers



Same module. Same command. The only difference is `test_error_paths.c`.

Test set	Tests	Lines	Branches	Functions
Happy paths only	14	82.1% (46/56)	77.3% (17/22)	**100%** (6/6)
+ error paths	23	**100%** (56/56)	**100%** (22/22)	100% (6/6)

The ten uncovered lines were exactly those guards, in the write path and the read path.



When FFF Cannot Reach

The `__syscall` Wall

Your module calls a sensor. There is no `sensor_sample_fetch_chan` symbol to replace:

```
sensor_sample_fetch_chan(dev, chan)        __syscall -> static inline
  └─ z_impl_sensor_sample_fetch_chan(...)  static inline, inlined into YOUR code
       └─ api->sample_fetch(dev, chan)     a function POINTER  <-- can be faked
```

```c
/* zephyr/include/zephyr/drivers/sensor.h */
static inline int z_impl_sensor_sample_fetch_chan(const struct device *dev,
                                                  enum sensor_channel type)
{
    const struct sensor_driver_api *api = (const struct sensor_driver_api *)dev->api;

    return api->sample_fetch(dev, type);
}
```



Faking a `struct device`



Build a device whose API table is made of FFF fakes:

```
DECLARE_FAKE_VALUE_FUNC(int, fake_sensor_sample_fetch,
                        const struct device *, enum sensor_channel);
DECLARE_FAKE_VALUE_FUNC(int, fake_sensor_channel_get,
                        const struct device *, enum sensor_channel, struct sensor_value *);

static const struct sensor_driver_api fake_sensor_api = {
    .sample_fetch = fake_sensor_sample_fetch,
    .channel_get  = fake_sensor_channel_get,
};

struct device fake_sensor_dev = {
    .name  = "fake_sensor",
    .api   = &fake_sensor_api,
    .state = &fake_sensor_state,     /* .initialized = 1, so device_is_ready() passes */
};
```



Other Mocking Tools



FFF Is Not the Only Option



Approach	Use it when
FFF	Default choice; you need call counts and arguments
`ztest_mock`	You want to set expected values before the call
Linker `--wrap`	You cannot rebuild the code, e.g. a prebuilt library
`zephyr,fake-*` drivers	The dependency is a standard driver (CAN, PWM, …)



Homework



- **Goal:** Write a fake yourself, for a module whose driver calls cannot be faked at link time
- **Module:** `temp_alarm` — reads a sensor, raises an alarm above a threshold
- **Starter code:** `tests/temp_alarm/` — `HOMEWORK.md`, `TEST_SPEC.md`, skeleton tests that build green
- **Tasks:**
  - Task 0 — finish `sensor_fake`: wire `sample_fetch` into the fake device
  - Task 1 — happy paths: below/above the threshold, reset clears state
  - Task 2 — error paths: `-EIO` propagates, `channel_get` is never reached, fail-then-recover
  - Task 3 — coverage before and after Task 2; commit `twister-out/coverage/`
- Push tags `l8-task0` … `l8-task3`

Thank You





Resources



1. Ztest — Mocking via FFF
2. FFF upstream repository
3. Ztest — Quick start: unit testing
4. Ztest — Shuffling test sequence
5. Ztest mocking API (`ztest_mock`)
6. Coverage reports using Twister
7. Twister
8. Bus emulators (`emul`)
9. Getting Started with Twister (Iomico)
10. Writing Your First C Tests with Ztest (Iomico)